Print copy of official Microsoft Learn documentation

Source: https://learn.microsoft.com/en-us/dotnet/core/extensions/configuration
Exported: 2026-10-14. Source metadata date: 2026-08-27T00:00:00Z.

Microsoft and documentation contributors. Article wording and code for the requested version are retained; web navigation is removed and print formatting is added. This locally generated PDF is not a Microsoft-published PDF book or a complete product manual. Source-page attribution and notices apply.

Documentation license and terms: Microsoft Learn terms; .NET documentation: CC BY 4.0 license. Check the source for specific attribution or exceptions.

Configuration in .NET

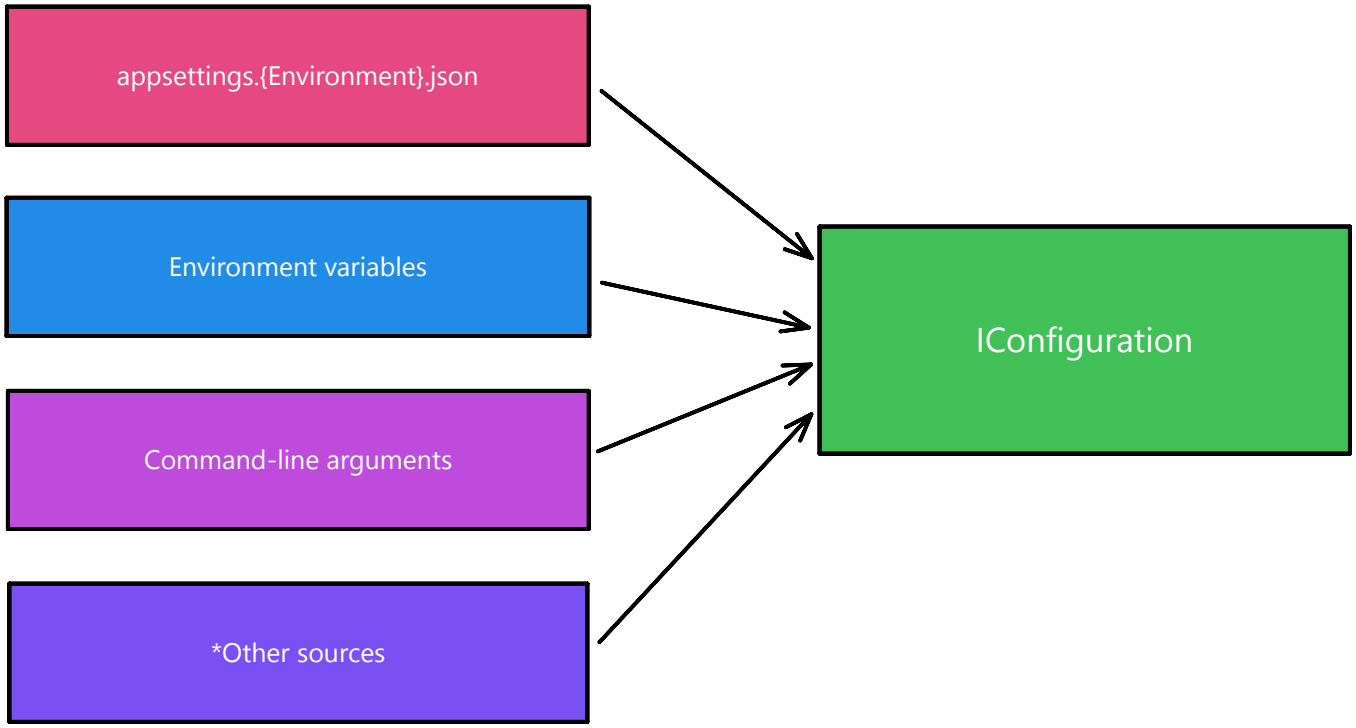
Configuration in .NET is performed using one or more configuration providers. Configuration providers read configuration data from key-value pairs using various configuration sources:

- Settings files, such as *appsettings.json*
- Environment variables
- Azure Key Vault
- Azure App Configuration
- Command-line arguments
- Custom providers (installed or created)
- Directory files
- In-memory .NET objects
- Third-party providers

> Note
>
> For information about configuring the .NET runtime itself, see .NET Runtime configuration settings.

Concepts and abstractions

Given one or more configuration sources, the IConfiguration type provides a unified view of the configuration data. Configuration is read-only, and the configuration pattern isn't designed to be programmatically writable. The `IConfiguration` interface is a single representation of all the configuration sources, as shown in the following diagram:



Configure console apps

.NET console apps created using the dotnet new command template or Visual Studio by default *don't* expose configuration capabilities. To add configuration in a new .NET console application, add a package reference to 📦 Microsoft.Extensions.Configuration. This package is the foundation for configuration in .NET apps. It provides the ConfigurationBuilder and related types.

```
using Microsoft.Extensions.Configuration;

var configuration = new ConfigurationBuilder()
    .AddInMemoryCollection(new Dictionary<string, string?>()
    {
        ["SomeKey"] = "SomeValue"
    })
    .Build();

Console.WriteLine(configuration["SomeKey"]);

// Outputs:
//   SomeValue
```

The preceding code:

- Creates a new ConfigurationBuilder instance.
- Adds an in-memory collection of key-value pairs to the configuration builder.
- Calls the Build() method to create an IConfiguration instance.
- Writes the value of the `SomeKey` key to the console.

While this example uses an in-memory configuration, there are many configuration providers available, exposing functionality for file-based, environment variables, command-line arguments, and other configuration sources. For more information, see Configuration providers in .NET.

Alternative hosting approach

Commonly, your apps will do more than just read configuration. They'll likely use dependency injection, logging, and other services. The .NET Generic Host approach is recommended for apps that use these services. Instead, consider adding a package reference to 📦 Microsoft.Extensions.Hosting. Modify the *Program.cs* file to match the following code:

```
using Microsoft.Extensions.Hosting;

using IHost host = Host.CreateApplicationBuilder(args).Build();

// Application code should start here.

await host.RunAsync();
```

The Host.CreateApplicationBuilder(String[]) method provides default configuration for the app in the following order, from highest to lowest priority:

1. Command-line arguments using the Command-line configuration provider.
2. Environment variables using the Environment Variables configuration provider.
3. App secrets when the app runs in the `Development` environment.
4. *appsettings.*`Environment`*.json* using the JSON configuration provider. For example, *appsettings.**Production**.json* and *appsettings.**Development**.json*.
5. *appsettings.json* using the JSON configuration provider.
6. ChainedConfigurationProvider : Adds an existing `IConfiguration` as a source.

Adding a configuration provider overrides previous configuration values. For example, the Command-line configuration provider overrides all values from other providers because it's added last. If `SomeKey` is set in both *appsettings.json* and the environment, the environment value is used because it was added after *appsettings.json*.

Binding

One of the key advantages of using the .NET configuration abstractions is the ability to *bind* configuration values to instances of .NET objects. For example, the JSON configuration provider can be used to map *appsettings.json* files to .NET objects and is used with dependency injection. This enables the options pattern, which uses classes to provide strongly typed access to groups of related settings. The default binder is reflection-based, but there's a source generator alternative that's easy to enable.

.NET configuration provides various abstractions. Consider the following interfaces:

- IConfiguration: Represents a set of key/value application configuration properties.
- IConfigurationRoot: Represents the root of an `IConfiguration` hierarchy.
- IConfigurationSection: Represents a section of application configuration values.

These abstractions are agnostic to their underlying configuration provider (IConfigurationProvider). In other words, you can use an `IConfiguration` instance to access any configuration value from multiple providers.

The binder processes configuration values in several ways:

- It uses a single configuration value directly for a `string` or `object` target, or converts it to a common .NET type, such as a numeric type, an enumeration, or a nullable type. The binder converts a Base64-encoded string to a byte array.
- It binds hierarchical configuration to an object's properties or constructor parameters.
- It binds child configuration sections to arrays, collections, sets, and dictionaries.
- It uses the matching configuration section directly when the target type is IConfigurationSection.
- It converts a single configuration value to a custom type. The default binder uses a TypeConverter when the custom type defines one. The source-generated binder doesn't support this conversion.

> Note
>
> The binder has a few limitations:
>
> - Properties are ignored if they have private setters or their type can't be converted.
> - Properties without corresponding configuration keys are ignored.

Binding hierarchies

Configuration values can contain hierarchical data. Hierarchical objects are represented with the use of the `:` delimiter in the configuration keys. To access a configuration value, use the `:` character to delimit a hierarchy. For example, consider the following configuration values:

```
{
  "Parent": {
    "FavoriteNumber": 7,
    "Child": {
      "Name": "Example",
      "GrandChild": {
        "Age": 3
      }
    }
  }
}
```

The following table represents example keys and their corresponding values for the preceding example JSON:

Key	Value
`"Parent:FavoriteNumber"`	`7`
`"Parent:Child:Name"`	`"Example"`
`"Parent:Child:GrandChild:Age"`	`3`

Advanced binding scenarios

The configuration binder has specific behaviors and limitations when working with certain types. This section includes the following subsections:

- Bind to dictionaries

- Dictionary keys with colons
- Bind to IReadOnly* types
- Exclude properties from binding
- Bind with parameterized constructors

Bind to dictionaries

When you bind configuration to a Dictionary<TKey,TValue> where the value is a mutable collection type (like arrays or lists), repeated binds to the same key extend the collection values instead of replacing them.

The following example demonstrates this behavior:

```
IConfiguration config = new ConfigurationBuilder()
    .AddInMemoryCollection()
    .Build();

config["Queue:0"] = "Value1";
var dict = new Dictionary<string, string[]>() { { "Queue", new[] { "InitialValue" } } };

Console.WriteLine("=== Dictionary Binding with Collection Values ===");
Console.WriteLine($"Initially: {string.Join(", ", dict["Queue"])}");

// In .NET 7+, binding extends the collection instead of replacing it.
config.Bind(dict);
Console.WriteLine($"After Bind: {string.Join(", ", dict["Queue"])}");

config["Queue:1"] = "Value2";
config.Bind(dict);
Console.WriteLine($"After 2nd Bind: {string.Join(", ", dict["Queue"])}");
```

For more information, see Binding config to dictionary extends values.

Dictionary keys with colons

The colon (`:`) character is reserved as a hierarchy delimiter in configuration keys. This means you can't use colons in dictionary keys when binding configuration. If your keys contain colons (such as URLs or other formatted identifiers), the configuration system interprets them as hierarchy paths rather than literal characters. Consider the following workarounds:

- Use alternative delimiter characters (such as double underscores `__`) in your configuration keys and transform them programmatically if needed.
- Manually deserialize the configuration as raw JSON using System.Text.Json or a similar library, which supports colons in keys.
- Create a custom mapping layer that translates safe keys to your desired keys with colons.

Bind to IReadOnly* types

The configuration binder doesn't support binding directly to `IReadOnlyList<T>`, `IReadOnlyDictionary<TKey, TValue>`, or other read-only collection interfaces. These interfaces lack the mechanisms the binder needs to populate the collections.

To work with read-only collections, use mutable types for the properties that the binder populates, then expose them as read-only interfaces for consumers:

```
Console.WriteLine("=== IReadOnly* Types (NOT Directly Supported) ===");

var readonlyConfig = new ConfigurationBuilder()
    .AddInMemoryCollection(new Dictionary<string, string?>
    {
        ["Settings:Values:0"] = "Item1",
        ["Settings:Values:1"] = "Item2",
        ["Settings:Values:2"] = "Item3",
    })
    .Build();

// This class uses List<string> for binding, exposes as IReadOnlyList<string>.
var settings = new SettingsWithReadOnly();
readonlyConfig.GetSection("Settings").Bind(settings);

Console.WriteLine("Values bound to mutable List, exposed as IReadOnlyList:");
foreach (var value in settings.ValuesReadOnly)
{
    Console.WriteLine($"  {value}");
}
```

The configuration class implementation:

```
class SettingsWithReadOnly
{
    // Use mutable type for binding
    public List<string> Values { get; set; } = [];

    // Expose as read-only for consumers
    public IReadOnlyList<string> ValuesReadOnly => Values;
}
```

This approach allows the binder to populate the mutable `List<string>` while presenting an immutable interface to consumers through `IReadOnlyList<string>`.

Exclude properties from binding

If you don't want the configuration binder to populate a property, apply ConfigurationIgnoreAttribute to it (available starting in .NET 11).

```
using Microsoft.Extensions.Configuration;

public sealed class AppOptions
{
    public string Endpoint { get; set; } = "";

    [ConfigurationIgnore]
    public string RuntimeValue { get; set; } = "calculated";
}
```

If configuration contains values for both `Endpoint` and `RuntimeValue`, the binder updates `Endpoint` and leaves `RuntimeValue` at its existing value.

Bind with parameterized constructors

Starting with .NET 7, the configuration binder supports binding to types with a single public parameterized constructor. This enables immutable types and records to be populated directly from configuration:

```
Console.WriteLine("=== Parameterized Constructor Binding ===");

var ctorConfig = new ConfigurationBuilder()
    .AddInMemoryCollection(new Dictionary<string, string?>
    {
        ["AppSettings:Name"] = "MyApp",
        ["AppSettings:MaxConnections"] = "100",
        ["AppSettings:Timeout"] = "30"
    })
    .Build();

// Binding to a type with a single parameterized constructor
var appSettings = ctorConfig.GetSection("AppSettings").Get<AppSettings>();
if (appSettings != null)
{
    Console.WriteLine($"Name: {appSettings.Name}");
    Console.WriteLine($"MaxConnections: {appSettings.MaxConnections}");
    Console.WriteLine($"Timeout: {appSettings.Timeout}");
}
```

The immutable settings class:

```
// Immutable type with single parameterized constructor.
class AppSettings
{
    public string Name { get; }
    public int MaxConnections { get; }
    public int Timeout { get; }

    public AppSettings(string name, int maxConnections, int timeout)
    {
        Name = name;
        MaxConnections = maxConnections;
        Timeout = timeout;
    }
}
```

Important

The binder only supports types with a single public parameterized constructor. If a type has multiple public parameterized constructors, the binder cannot determine which one to use and binding will fail. Use either a single parameterized constructor or a parameterless constructor with property setters.

Basic example

To access configuration values in their basic form, without the assistance of the *generic host* approach, use the ConfigurationBuilder type directly.

Tip

The System.Configuration.ConfigurationBuilder type is different to the Microsoft.Extensions.Configuration.ConfigurationBuilder type. All of this content is specific to the `Microsoft.Extensions.*` NuGet packages and namespaces.

Consider the following C# project:

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>true</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <Content Include="appsettings.json">
      <CopyToOutputDirectory>Always</CopyToOutputDirectory>
    </Content>
  </ItemGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.Extensions.Configuration.Binder" Version="10.0.12" />
    <PackageReference Include="Microsoft.Extensions.Configuration.Json" Version="10.0.12" />
    <PackageReference Include="Microsoft.Extensions.Configuration.EnvironmentVariables" Version="10.0.12"
/>
  </ItemGroup>

</Project>
```

The preceding project file references several configuration NuGet packages:

- Microsoft.Extensions.Configuration.Binder: Functionality to bind an object to data in configuration providers for `Microsoft.Extensions.Configuration`.
- Microsoft.Extensions.Configuration.Json: JSON configuration provider implementation for `Microsoft.Extensions.Configuration`.
- Microsoft.Extensions.Configuration.EnvironmentVariables: Environment variables configuration provider implementation for `Microsoft.Extensions.Configuration`.

Consider an example *appsettings.json* file:

```
{
    "Settings": {
        "KeyOne": 1,
        "KeyTwo": true,
        "KeyThree": {
            "Message": "Oh, that's nice...",
            "SupportedVersions": {
                "v1": "1.0.0",
                "v3": "3.0.7"
            }
        },
        "IPAddressRange": [
            "46.36.198.121",
            "46.36.198.122",
            "46.36.198.123",
            "46.36.198.124",
```

```
            "46.36.198.125"
        ]
    }
}
```

Now, given this JSON file, here's an example consumption pattern using the configuration builder directly:

```
using Microsoft.Extensions.Configuration;

// Build a config object, using env vars and JSON providers.
IConfigurationRoot config = new ConfigurationBuilder()
    .AddJsonFile("appsettings.json")
    .AddEnvironmentVariables()
    .Build();

// Get values from the config given their key and their target type.
Settings? settings = config.GetRequiredSection("Settings").Get<Settings>();

// Write the values to the console.
Console.WriteLine($"KeyOne = {settings?.KeyOne}");
Console.WriteLine($"KeyTwo = {settings?.KeyTwo}");
Console.WriteLine($"KeyThree:Message = {settings?.KeyThree?.Message}");

// Application code which might rely on the config could start here.

// This will output the following:
//   KeyOne = 1
//   KeyTwo = True
//   KeyThree:Message = Oh, that's nice...
```

The preceding C# code:

- Instantiates a ConfigurationBuilder.
- Adds the `"appsettings.json"` file to be recognized by the JSON configuration provider.
- Adds environment variables as being recognized by the Environment Variable configuration provider.
- Gets the required `"Settings"` section and the corresponding `Settings` instance by using the `config` instance.

The `Settings` object is shaped as follows:

```
public sealed class Settings
{
    public required int KeyOne { get; set; }
    public required bool KeyTwo { get; set; }
    public required NestedSettings KeyThree { get; set; } = null!;
}
```

```
public sealed class NestedSettings
{
    public required string Message { get; set; } = null!;
}
```

Basic example with hosting

To access the `IConfiguration` value, you can rely again on the `Microsoft.Extensions.Hosting` NuGet package. Create a new console application, and paste the following project file contents into it:

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>true</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <Content Include="appsettings.json">
      <CopyToOutputDirectory>Always</CopyToOutputDirectory>
    </Content>
  </ItemGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.Extensions.Configuration.Binder" Version="10.0.12" />
    <PackageReference Include="Microsoft.Extensions.Hosting" Version="10.0.12" />
  </ItemGroup>

</Project>
```

The preceding project file defines that:

- The application is an executable.
- An *appsettings.json* file is to be copied to the output directory when the project is compiled.
- The `Microsoft.Extensions.Hosting` NuGet package reference is added.

Add the *appsettings.json* file at the root of the project with the following contents:

```
{
    "KeyOne": 1,
    "KeyTwo": true,
    "KeyThree": {
        "Message": "Thanks for checking this out!"
    }
}
```

Replace the contents of the *Program.cs* file with the following C# code:

```
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;

using IHost host = Host.CreateApplicationBuilder(args).Build();

// Ask the service provider for the configuration abstraction.
IConfiguration config = host.Services.GetRequiredService<IConfiguration>();

// Get values from the config given their key and their target type.
int keyOneValue = config.GetValue<int>("KeyOne");
bool keyTwoValue = config.GetValue<bool>("KeyTwo");
```

```
string? keyThreeNestedValue = config.GetValue<string>("KeyThree:Message");

// Write the values to the console.
Console.WriteLine($"KeyOne = {keyOneValue}");
Console.WriteLine($"KeyTwo = {keyTwoValue}");
Console.WriteLine($"KeyThree:Message = {keyThreeNestedValue}");

// Application code which might rely on the config could start here.

await host.RunAsync();

// This will output the following:
//   KeyOne = 1
//   KeyTwo = True
//   KeyThree:Message = Thanks for checking this out!
```

When you run this application, the `Host.CreateApplicationBuilder` defines the behavior to discover the JSON configuration and expose it through the `IConfiguration` instance. From the `host` instance, you can ask the service provider for the `IConfiguration` instance and then ask it for values.

> Tip
>
> Using the raw `IConfiguration` instance in this way, while convenient, doesn't scale very well. When applications grow in complexity, and their corresponding configurations become more complex, we recommend that you use the *options pattern* as an alternative.

Basic example with hosting and using the indexer API

Consider the same *appsettings.json* file contents from the previous example:

```
{
    "SupportedVersions": {
        "v1": "1.0.0",
        "v3": "3.0.7"
    },
    "IPAddressRange": [
        "46.36.198.123",
        "46.36.198.124",
        "46.36.198.125"
    ]
}
```

Replace the contents of the *Program.cs* file with the following C# code:

```
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;

using IHost host = Host.CreateApplicationBuilder(args).Build();

// Ask the service provider for the configuration abstraction.
IConfiguration config = host.Services.GetRequiredService<IConfiguration>();

// Get values from the config given their key and their target type.
```

```
string? ipOne = config["IPAddressRange:0"];
string? ipTwo = config["IPAddressRange:1"];
string? ipThree = config["IPAddressRange:2"];
string? versionOne = config["SupportedVersions:v1"];
string? versionThree = config["SupportedVersions:v3"];

// Write the values to the console.
Console.WriteLine($"IPAddressRange:0 = {ipOne}");
Console.WriteLine($"IPAddressRange:1 = {ipTwo}");
Console.WriteLine($"IPAddressRange:2 = {ipThree}");
Console.WriteLine($"SupportedVersions:v1 = {versionOne}");
Console.WriteLine($"SupportedVersions:v3 = {versionThree}");

// Application code which might rely on the config could start here.

await host.RunAsync();

// This will output the following:
//     IPAddressRange:0 = 46.36.198.123
//     IPAddressRange:1 = 46.36.198.124
//     IPAddressRange:2 = 46.36.198.125
//     SupportedVersions:v1 = 1.0.0
//     SupportedVersions:v3 = 3.0.7
```

The values are accessed using the indexer API where each key is a string, and the value is a string. Configuration supports properties, objects, arrays, and dictionaries.

Configuration providers

The following table shows the configuration providers available to .NET Core apps.

Configuration provider	Provides configuration from
Azure App Configuration	Azure App Configuration
Azure Key Vault	Azure Key Vault
Command-line	Command-line parameters
Custom	Custom source
Environment variables	Environment variables
File	JSON, XML, and INI files
Key-per-file	Directory files
Memory	In-memory collections
App secrets (secret manager)	File in the user profile directory

Tip

The order in which configuration providers are added matters. When multiple configuration providers are used and more than one provider specifies the same key, the last one added is used.

For more information on various configuration providers, see Configuration providers in .NET.

See also

- Configuration providers in .NET
- Implement a custom configuration provider
- Configuration bugs should be created in the github.com/dotnet/runtime repo
- Configuration in ASP.NET Core